

3장 순차

1. 조합칩

*조합칩 : 불논리칩/산술칩의 조합

-입력값의 조합에만 의존하는 함수

-상태를 유지하지 않는다

*메모리소자는 순차칩으로 만든다. (sequential chip)

*플립플롭

데이터플립플롭을 DFF라고 한다. (1비트 입력/ 1비트 출력)

클럭입력신호도 받는다

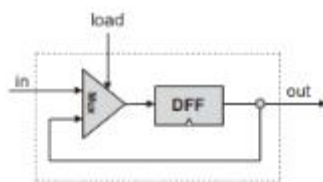
DFF의 동작은 단순히 이전 시간의 입력을 출력한다.

2진셀, 레지스터, RAM 까지 컴퓨터에서 상태를 유지하는 모든 하드웨어의 기초이다.



$$\text{out}(t) = \text{in}(t-1)$$

Flip-flop



if load(t-1) then out(t) = in(t-1)
else out(t) = out(t-1)

1-bit register (Bit)

*멀티비트값을 저장하는 레지스터를 만들려면 1비트 레지스터를 필요한 만큼 배열한다

*레지스터가 저장할 수 있는 비트의 개수를 폭(Width)

*레지스터에 저장되는 멀티 비트값은 Word라 부른다.

*메모리

임의 길이의 메모리 뱅크 (RAM 같은것)

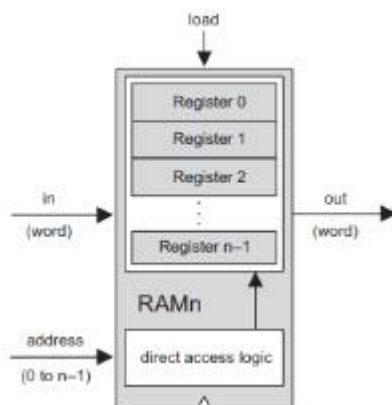
임의접근 메모리는 접근 순서와 관계없이 무작위로 선택된 Word를 읽고 쓸 수 있다.

메모리 내의 어떤 단어든 물리적 저장위치와 관계없이 똑같은 속도로 직접 접근 가능하다.

직접 접근 논리라는 것을 이용해서 레지스터내의 특정 Word에 접근한다. 주소같은 것

RAM 기본변수로는 Word의 비트 수를 뜻하는 Width, Word 수 나타내는 Size가 있다.

컴퓨터는 데이터 Width가 32/64비트이다.

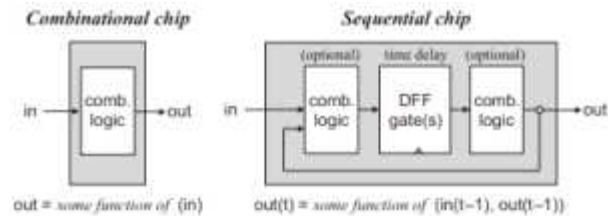


2. 계수기

*계수기 : 매시간 단위마다 내부 상태값을 증가시키는 순차칩

ex) 프로그램 계수기 : 다음에 실행해야할 프로그램 명령의 주소를 출력하는 기능

-기능 : 0으로 값을 맞추는 기능/새로운 계수값 로드/값 감소 기능



*조합칩의 출력은 시간과 무관하게 입력이 바뀔때만 변경된다.

*DFF가 들어간 순차칩의 출력은 한 클럭 사이클이 넘어갈때 바뀌며 한 사이클안에서는 바뀌지 않는다.

*클럭 사이클 중간에서는 순차칩의 상태가 불안정해도 상관이 없다.

다음 사이클을 시작하는 순간에만 올바른 값을 출력하면 된다.

*하나의 컴퓨터 아키텍처 내에서 가장 긴 경로를 따라 전송되는 시간보다 클럭 사이클 주기를 살짝 더 길게 만들면 된다.

조합 칩이 자신의 상태를 바꾸기 전까지 ALU에서 전송받는 입력값이 유효함을 보장할 수 있다.

<실습>

*Clock

Clock: Simulated implementation



Interactive simulation

A clock icon, used to generate a sequence of tick-tock signals:

0, 0+, 1, 1+, 2, 2+, 3, 3+, ...

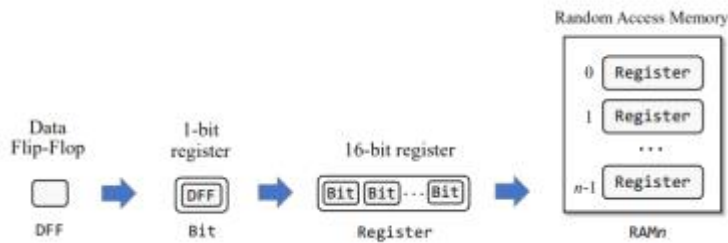


Script-based simulation

"tick" and "tock" commands, used to advance the clock:

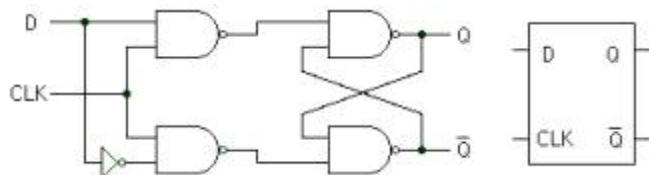
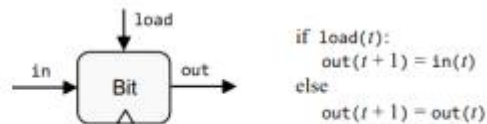
```
...
// Sets inputs, advances the clock, and
// writes output values as it goes along.
set in 19,
set load 1,
tick,
output,
tock,
output,
tick, tock,
output,
...
```

0. 개요도



1. 1비트 레지스터

1-Bit register



이거는 플립플롭이다. 1 bit register 만들려면 DFF와 MUX로 해야한다.
이 프로젝트에서는 DFF는 만들지 않고 builtIn 코드 사용한다

CHIP Bit {

IN in, load;

OUT out;

PARTS:

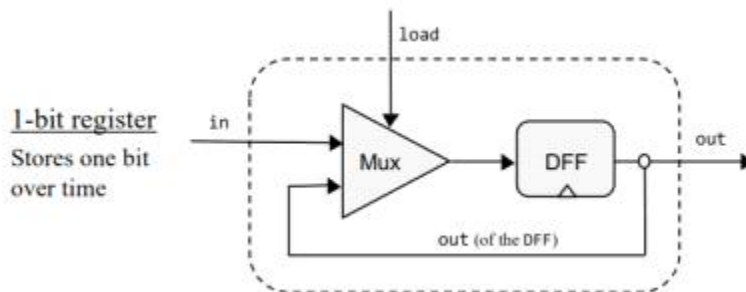
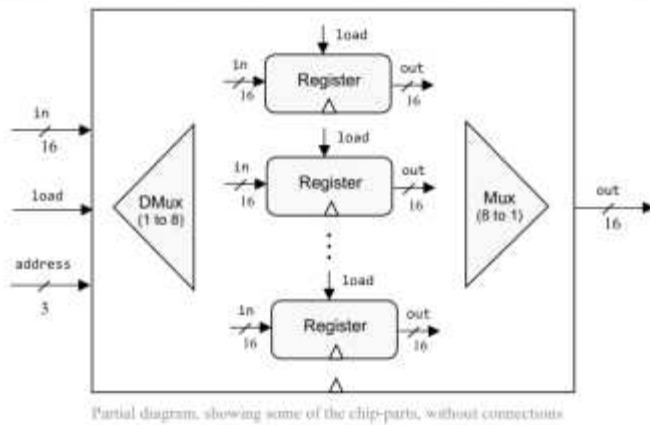
Mux(a=outDf, b=in, sel=load,out=muxOut);

DFF(in=muxOut, out=out, out=outDf);

}

DFF 라는 것을 잘 봐야한다. Mux의 입력값 새로 들어온 값이나 기존 값 중 하나가 출력으로 나오는 것이므로 상태를 유지하려면 피드백회로 구성을 해야한다 .

8-Register RAM



2. 레지스터

CHIP Register {
IN in[16], load;
OUT out[16];

PARTS:

//// Replace this comment with your code.

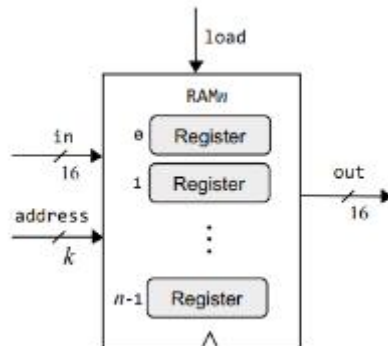
```

Bit(in=in[0], load=load, out=out[0]);
Bit(in=in[1], load=load, out=out[1]);
Bit(in=in[2], load=load, out=out[2]);
Bit(in=in[3], load=load, out=out[3]);
Bit(in=in[4], load=load, out=out[4]);
Bit(in=in[5], load=load, out=out[5]);
Bit(in=in[6], load=load, out=out[6]);
Bit(in=in[7], load=load, out=out[7]);
Bit(in=in[8], load=load, out=out[8]);
Bit(in=in[9], load=load, out=out[9]);
Bit(in=in[10], load=load, out=out[10]);
Bit(in=in[11], load=load, out=out[11]);
Bit(in=in[12], load=load, out=out[12]);
Bit(in=in[13], load=load, out=out[13]);
Bit(in=in[14], load=load, out=out[14]);
Bit(in=in[15], load=load, out=out[15]);

```

}

3. RAM8



```
CHIP RAM8 {
    IN in[16], load, address[3];
    OUT out[16];

    PARTS:
        DMux8Way(in=load, sel=address, a=asel, b=bsel, c=csel, d=dsel, e=esel, f=fsel, g=gsel, h=hsel);
        Register(in=in, load=asel, out=outNode1);
        Register(in=in, load=bsel, out=outNode2);
        Register(in=in, load=csel, out=outNode3);
        Register(in=in, load=dsel, out=outNode4);
        Register(in=in, load=esel, out=outNode5);
        Register(in=in, load=fsel, out=outNode6);
        Register(in=in, load=gsel, out=outNode7);
        Register(in=in, load=hsel, out=outNode8);
        Mux8Way16(a=outNode1, b=outNode2, c=outNode3, d=outNode4,
e=outNode5, f=outNode6, g=outNode7, h=outNode8, sel=address, out=out);
}
```

4. RAM64

```
CHIP RAM64 {
    IN in[16], load, address[6];
    OUT out[16];

    PARTS:
        //address[0..2] points to Register in a RAM8
        //address[3..5] points to RAM8 in a RAM64
        //select RAM8
        DMux8Way(in=load, sel=address[3..5], a=aa, b=bb, c=cc, d=dd, e=ee, f=ff, g=gg, h=hh);
        //inner
        RAM8(in=in, address=address[0..2], load=aa, out=a1);
        RAM8(in=in, address=address[0..2], load=bb, out=a2);
        RAM8(in=in, address=address[0..2], load=cc, out=a3);
        RAM8(in=in, address=address[0..2], load=dd, out=a4);
        RAM8(in=in, address=address[0..2], load=ee, out=a5);
        RAM8(in=in, address=address[0..2], load=ff, out=a6);
        RAM8(in=in, address=address[0..2], load=gg, out=a7);
        RAM8(in=in, address=address[0..2], load=hh, out=a8);
        //out part
        Mux8Way16(a=a1, b=a2, c=a3, d=a4, e=a5, f=a6, g=a7, h=a8, sel=address[3..5], out=out);
}
```

5. RAM-n

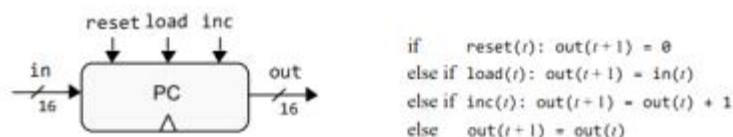
```
CHIP RAM512 (
  IN in[16], load, address[9];
  OUT out[16];

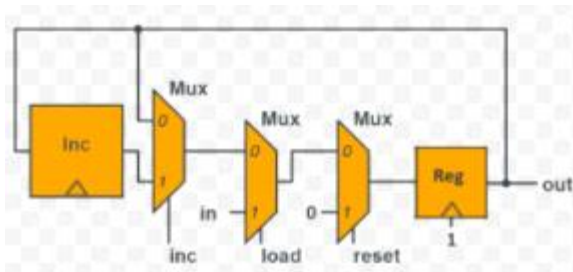
  PARTS:
    //address[0..2] points to Register in a RAM8
    //address[3..5] points to RAM8 in a RAM64
    //address[6..8] points to RAM64 in a RAM512
    //select RAM64
    DMux8Way(in=load, sel=address[6..8], a=aa,b=bb, c=cc,d=dd,e=ee,f=ff,g=gg, h=hh);
    //inner
    RAM64(in=in , address=address[0..5] , load=aa ,out=a1);
    RAM64(in=in , address=address[0..5] , load=bb ,out=a2);
    RAM64(in=in , address=address[0..5] , load=cc ,out=a3);
    RAM64(in=in , address=address[0..5] , load=dd ,out=a4);
    RAM64(in=in , address=address[0..5] , load=ee ,out=a5);
    RAM64(in=in , address=address[0..5] , load=ff ,out=a6);
    RAM64(in=in , address=address[0..5] , load=gg ,out=a7);
    RAM64(in=in , address=address[0..5] , load=hh ,out=a8);
    //out part
    Mux8Way16(a=a1, b=a2, c=a3, d=a4, e=a5, f=a6, g=a7, h=a8, sel=address[6..8], out=out);
}

CHIP RAM4K (
  IN in[16], load, address[12];
  OUT out[16];

  PARTS:
    //address[0..2] points to Register in a RAM8
    //address[3..5] points to RAM8 in a RAM64
    //address[6..8] points to RAM64 in a RAM512
    //address[9..11] points to RAM512 in a RAM4k
    //select RAM512
    DMux8Way(in=load, sel=address[9..11], a=aa,b=bb, c=cc,d=dd,e=ee,f=ff,g=gg, h=hh);
    //inner
    RAM512(in=in , address=address[0..8] , load=aa ,out=a1);
    RAM512(in=in , address=address[0..8] , load=bb ,out=a2);
    RAM512(in=in , address=address[0..8] , load=cc ,out=a3);
    RAM512(in=in , address=address[0..8] , load=dd ,out=a4);
    RAM512(in=in , address=address[0..8] , load=ee ,out=a5);
    RAM512(in=in , address=address[0..8] , load=ff ,out=a6);
    RAM512(in=in , address=address[0..8] , load=gg ,out=a7);
    RAM512(in=in , address=address[0..8] , load=hh ,out=a8);
    //out part
    Mux8Way16(a=a1, b=a2, c=a3, d=a4, e=a5, f=a6, g=a7, h=a8, sel=address[9..11], out=out);
}
```

6. PC : Programming Counter





```
CHIP PC {
  IN in[16],inc, load, reset;
  OUT out[16];
```

PARTS:

//// Replace this comment with your code

```
Inc16(in=nodeOut ,out=nodeInc);
```

```
Mux16(a=nodeOut,b=nodeInc,sel=inc, out=nodeLoad);
```

```
Mux16(a=nodeLoad, b=in,sel=load,out=nodeReset);
```

```
Mux16(a=nodeReset,b=false, sel=reset, out=nodeReg);
```

```
Register(in=nodeReg,load=true,out=nodeOut, out=out);
```

```
}
```